DAY 07 · THEORY HANDOUT

KNN: Learn From Nearby Examples

Compare a new input with nearby labeled examples and choose a class by voting.

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

02 KGP Talkie

A new fruit needs a class

There are six fruits with known labels. A new fruit weighs 164 g.

Fruit weight: feature	Known fruit: target
120 g	apple
130 g	apple
160 g	apple
170 g	orange
180 g	orange
190 g	orange

Example records. The new fruit weighs 164 g; its class is unknown.

K-Nearest Neighbors (KNN) finds nearby training records and uses their answers to make a prediction.

The **feature** is fruit weight. The **target** is the known class: apple or orange.

KNN stores labeled training examples. It does not learn one line of weights. Most of its work happens when a new input needs a prediction.

> Weight alone cannot identify every fruit. The records show how the method works with one feature.

Step 1: calculate distance

With one numeric feature, distance is the size of the difference between two values.

Stored weight	Difference from 164 g	Distance
120 g	120 - 164 = -44	44 g
130 g	130 - 164 = -34	34 g
160 g	160 - 164 = -4	4 g
170 g	170 - 164 = 6	6 g
180 g	180 - 164 = 16	16 g
190 g	190 - 164 = 26	26 g

Compare the new weight, **164 g**, with every stored weight. A 160 g apple is 4 g away. A 170 g orange is 6 g away.

Ignore the sign of the difference. A negative difference is not a negative distance.

Smaller distance means a closer input value. With several numeric features, KNN can use straight-line distance across those features.

Every distance in this example is measured using the same feature and unit: grams.

Step 2: pick the K nearest

Sort records from the smallest distance to the largest distance.

Distance order	Stored weight	Class	Distance
1	160 g	apple	4 g
2	170 g	orange	6 g
3	180 g	orange	16 g
4	190 g	orange	26 g
5	130 g	apple	34 g
6	120 g	apple	44 g

For K = 3, keep only the first three records in this sorted list.

K = 3 means three neighbors will take part in the prediction. K is a setting chosen before prediction.

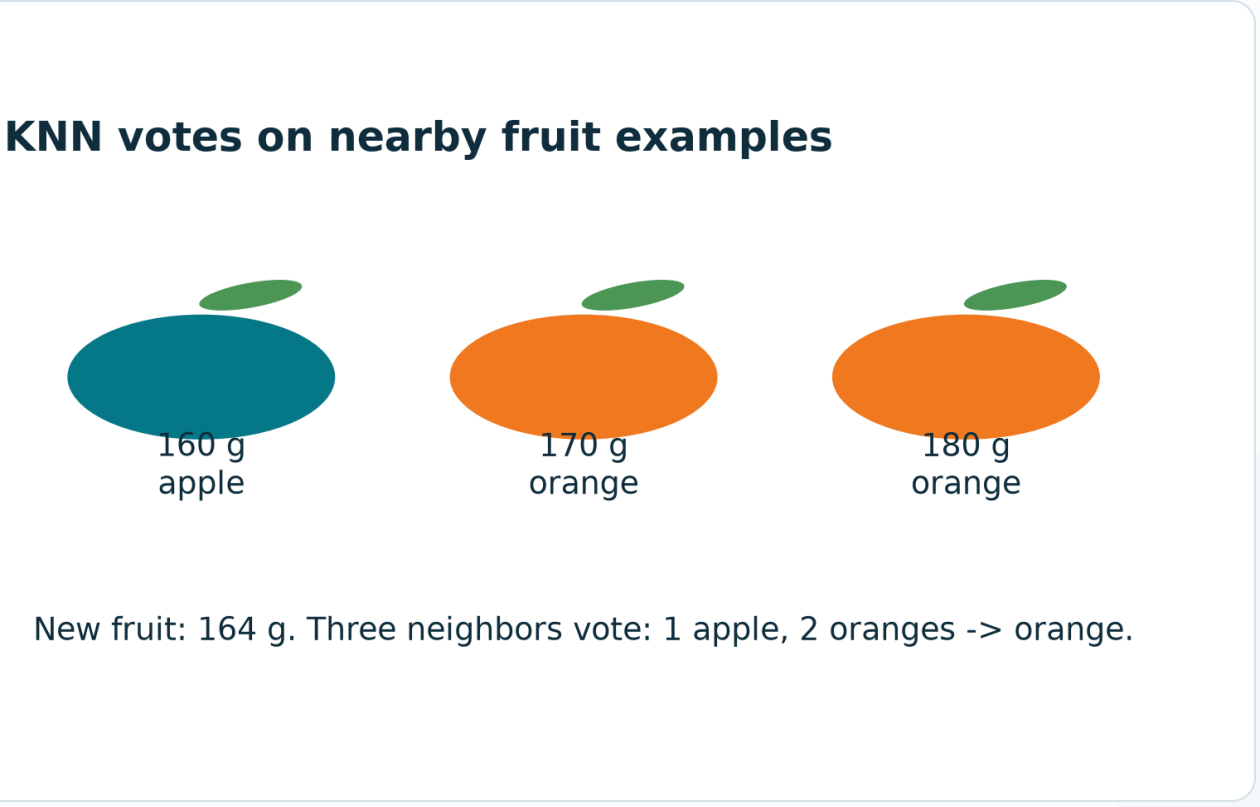
The nearest records are **160 g apple**, **170 g orange**, and **180 g orange**. Their distances are 4, 6, and 16 g.

The other three records are farther away. They do not vote when K is 3.

Nearest means nearest by the selected feature values, not nearest by row number or name.

Step 3: count the votes

Each of the three selected neighbors gets one vote.



Apple receives **1 vote**. Orange receives **2 votes**. The most common class is orange.

The new 164 g fruit is therefore **predicted as orange**. The closest record alone was an apple, but the group vote was orange.

This is **uniform voting**: every selected neighbor counts equally. Another option gives closer neighbors more weight.

A vote gives a prediction. The true fruit label is needed to know whether the prediction was right.

Step 4: understand the effect of K

K changes how many nearby examples influence the answer.

K	Records that vote	Prediction
1	160 g: apple	apple
3	160 g: apple; 170 g: orange; 180 g: orange	orange

Same new fruit: 164 g. A different K can give a different class.

Small K follows very local details. One unusual or wrongly labeled record can change the answer.

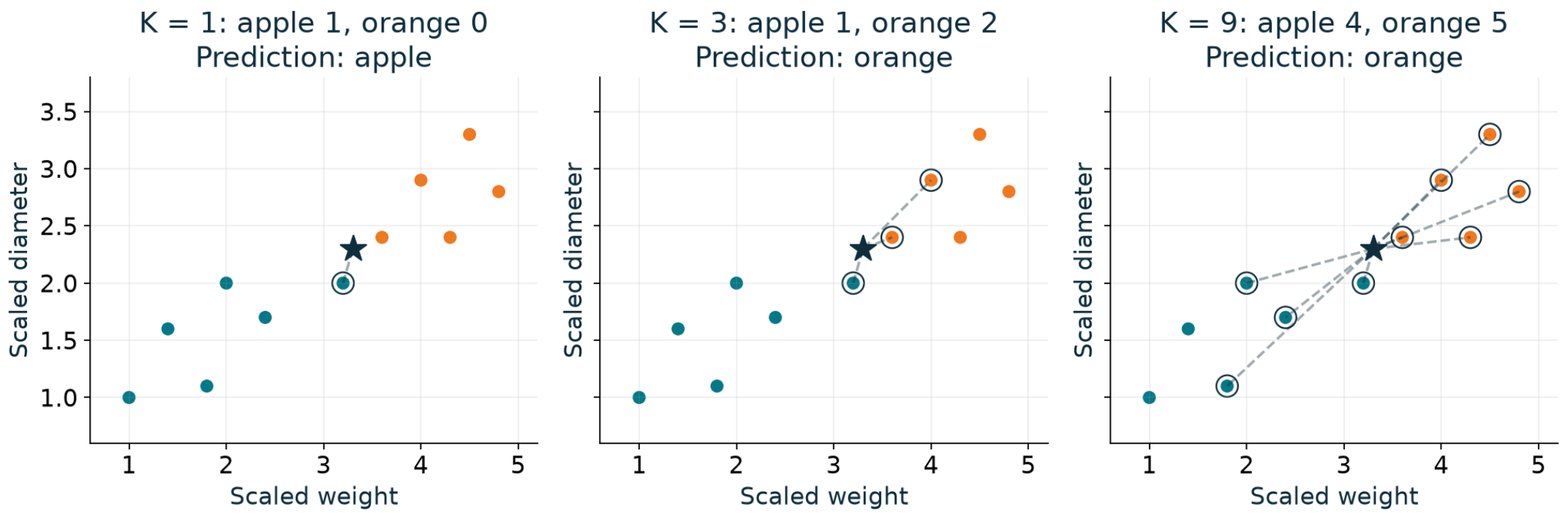
Larger K uses a wider group. It may mix different classes and miss a local pattern.

Hyperparameter means a setting chosen outside the fitted data values. K is a hyperparameter. A validation set can help choose it without using final test answers.

An odd K avoids an equal vote count between two classes with uniform voting. No K is best for every dataset.

The same fruit with different K values

Teal points are labeled apples. Orange points are labeled oranges. The star is a new fruit.



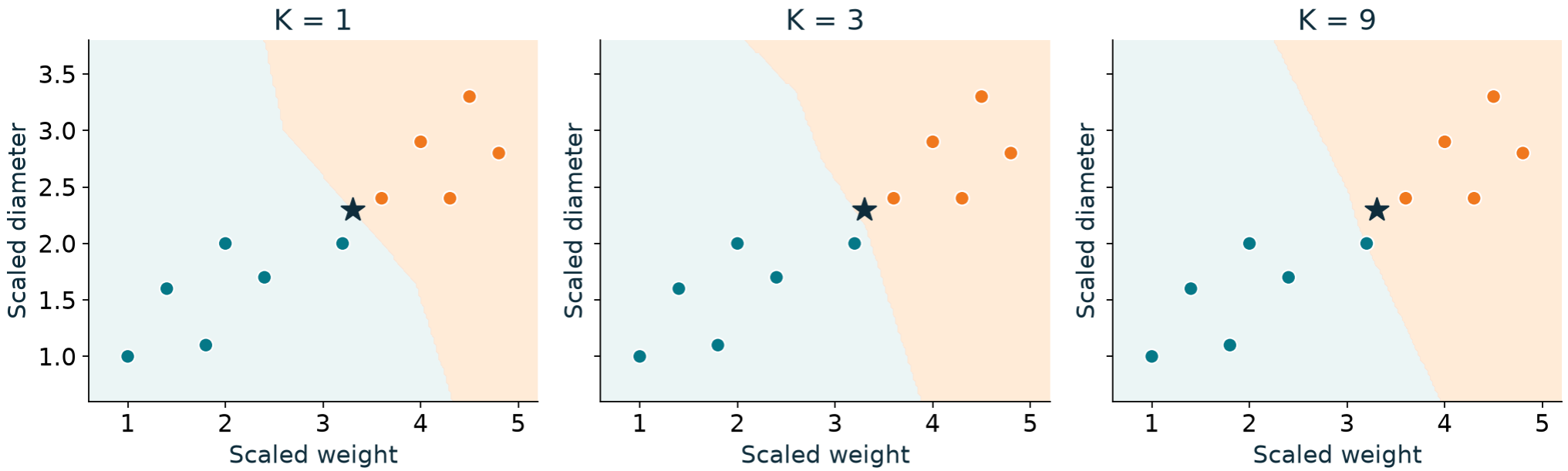
Rings mark the selected neighbors. Their labels vote: **K=1 gives apple**; **K=3 and K=9 give orange**.

> Both feature scales are comparable in this illustration. No K is best for every dataset.

See the prediction regions

Every background position is given the class that KNN would predict there.

Colored regions show which class KNN predicts at each position

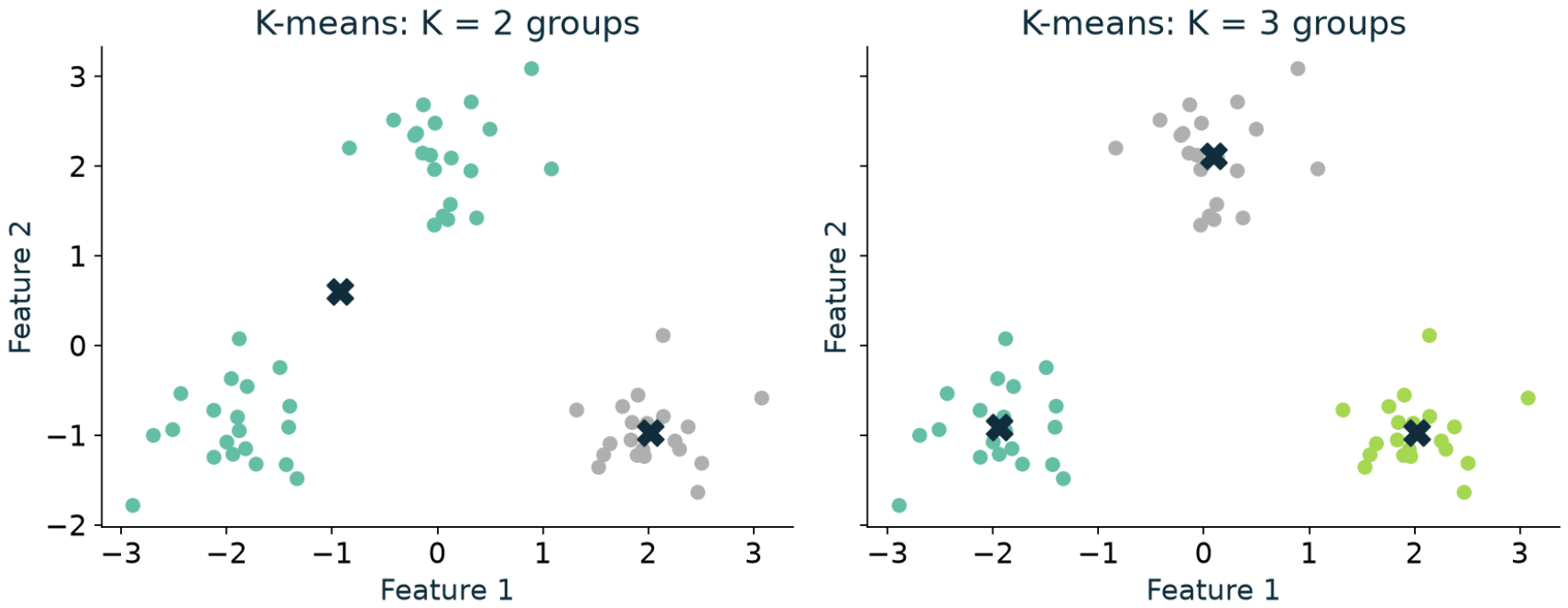


Light teal means predicted apple. Light orange means predicted orange. The same training labels are used in all three pictures.

> These are class prediction regions. KNN does not learn new clusters from unlabeled points.

KNN and K-means use K differently

A cluster is a group of similar records. A class is a known label.



KNN: K counts neighbors that vote using known labels. **K-means:** K counts groups formed without known labels. X marks a group center.

> K-means is a different algorithm. Two groups may merge when K changes from 3 to 2.

When several features need scaling

With weight and diameter together, raw number ranges can change which records look close.

Feature	Example range	Possible effect
Weight in grams	120 to 190	Large number differences
Diameter in cm	5 to 9	Small number differences

Scaling puts feature ranges on comparable scales.
Learn the scale from training records, then reuse it for new records.

A 20 g weight difference is numerically much larger than a 1 cm diameter difference. Without scaling, weight may dominate the distance.

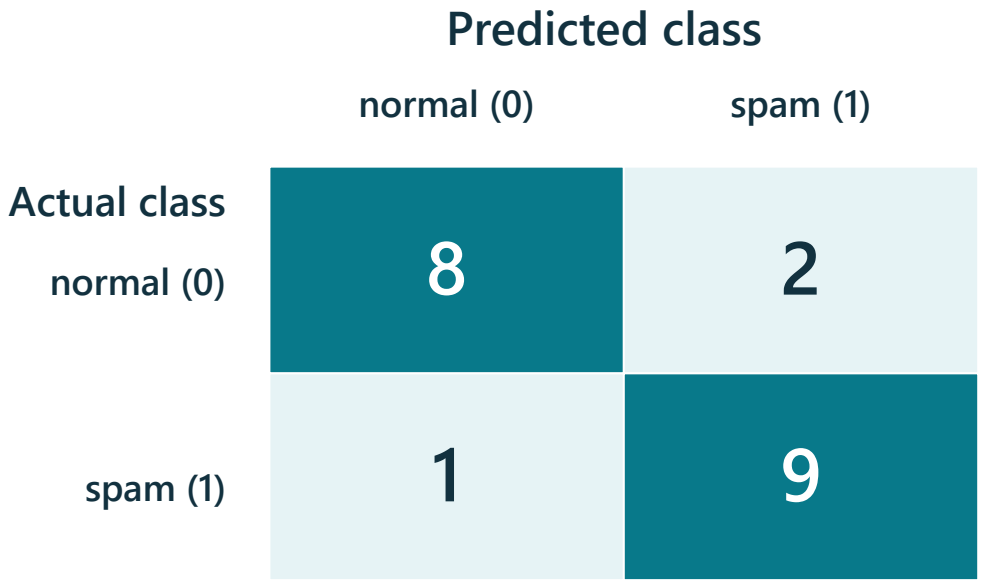
Feature scaling puts ranges on comparable scales. The rule is learned using training records only.

The same learned rule must be used for test records and new input. With only weight in grams, all distances already use the same quantity.

Distance, feature choice, scaling, and K all affect a KNN prediction.

A confusion matrix counts mistakes

A spam filter predicts 20 messages. Spam is the positive class, written 1.



True Positive (TP) = 9: spam correctly flagged. **True Negative (TN) = 8:** normal email correctly left alone.

False Positive (FP) = 2: normal email wrongly flagged. **False Negative (FN) = 1:** spam that was missed.

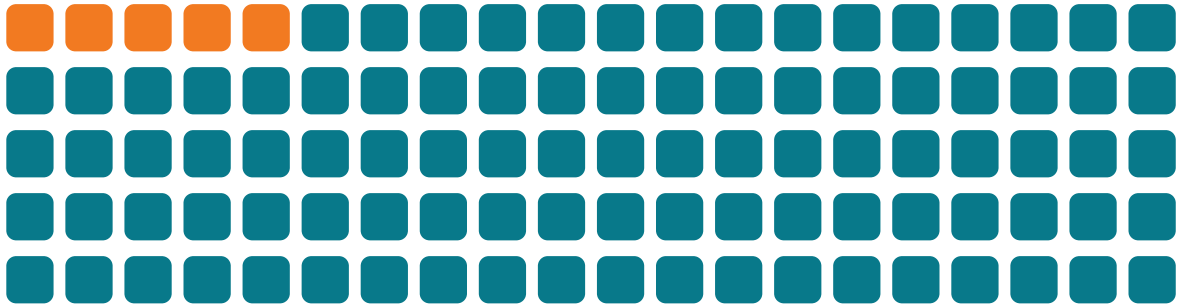
Rows show actual classes. Columns show predicted classes. The diagonal holds correct predictions.

> Positive means the class being counted as 1. It does not mean good or safe.

Accuracy: how many were right?

Accuracy counts correct predictions across all classes.

100 messages: 95 normal, 5 spam



Always say normal: 95% accuracy, 0% spam recall

In the 20-message matrix, 8 normal emails and 9 spam messages were right. **Accuracy = 17 / 20 = 85%.**

Use accuracy when class sizes and mistake costs are fairly similar. Also read the confusion matrix.

With 95 normal messages and only 5 spam messages, always saying normal gives 95% accuracy but finds no spam. High accuracy can hide this failure.

Class imbalance means one class has many more records than another.

Precision and recall: which mistake matters?

The same model can have two different scores because each metric counts a different group.

Metric	Question	Same 20-message example
Precision	Were positive calls right?	9 real spam / 11 spam calls = 81.8%
Recall	Were real positives found?	9 found / 10 real spam = 90%

Precision checks the predicted positive group.
Recall checks the actual positive group.

Use precision when wrong positive calls are costly. A spam filter should not hide important normal email by mistake.

Use recall when missing positive cases is costly. If many real spam messages reach the inbox, check how much spam was missed.

Another recall example is finding broken items before packing. Out of 10 broken items, finding 9 gives recall = 90%. The positive class there is broken.

> Precision asks "Were the flags right?" Recall asks "Did the model find the real cases?"

F1 and choosing a metric

Choose the metric by the cost of a wrong call or a missed case.

Measure	What matters	Example use
Accuracy	All correct calls	Similar class sizes and mistake costs
Precision	Wrong positive calls	Avoid hiding normal email as spam
Recall	Missed positive cases	Find more of the actual spam
F1	Precision and recall together	Both wrong flags and missed cases matter

For the same matrix: F1 = 2 x 9 / (2 x 9 + 2 + 1) = 0.857

F1 score combines precision and recall. It is their harmonic mean: an average that stays low when either one is low.

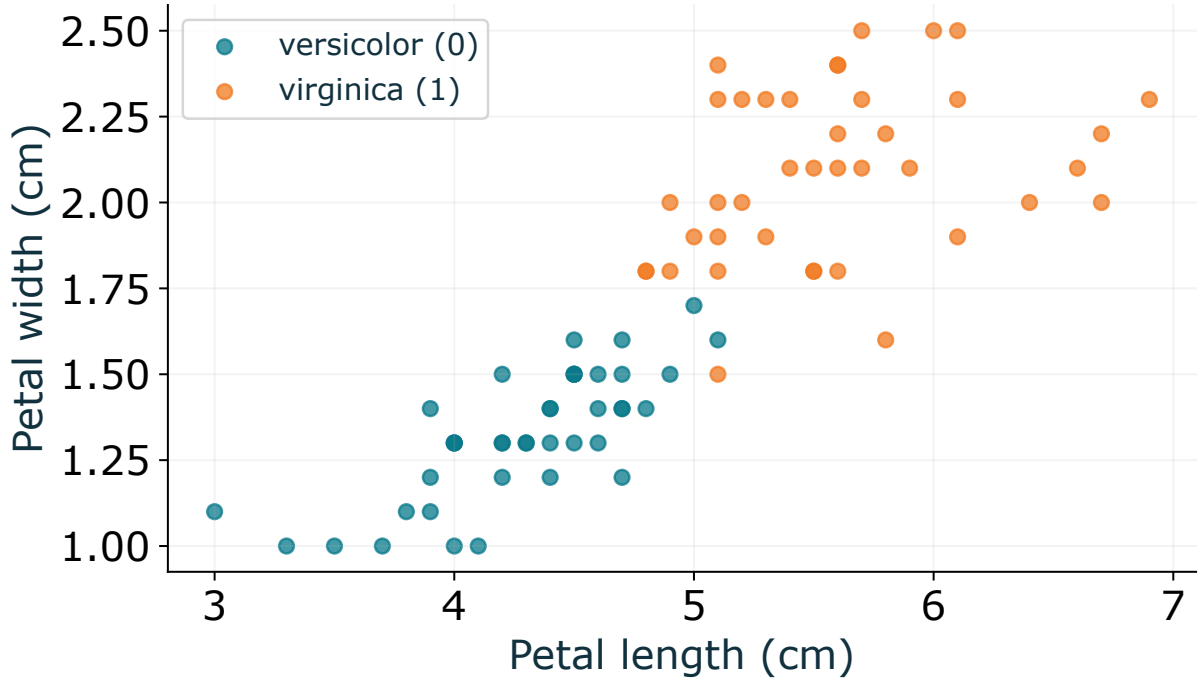
Use F1 when both wrong positive calls and missed positive cases matter. F1 does not count true negatives.

A single score does not choose the best model for every task. Start with the confusion matrix and decide which mistake matters most.

For the 20-message example, precision is 81.8%, recall is 90%, and F1 is about 85.7%.

An example with flower measurements

The iris data contains measured petals and known species names.



This example selects 100 flowers from two species. The **features** are petal length and width. A petal is a colored part of a flower.

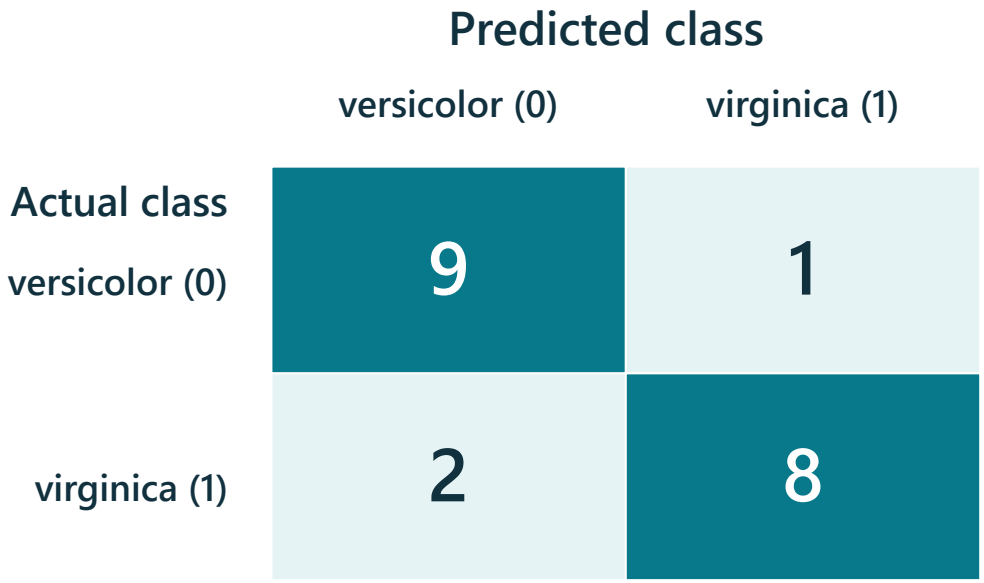
The **target** is species: 0 is versicolor and 1 is virginica. The split has 80 training flowers and 20 held-out test flowers.

The feature values are scaled using training data so their ranges are comparable. The same learned scale is then applied to new input.

The classes overlap in some measurements. A model can still confuse one species with the other.

Read the held-out predictions

KNN predicts 20 flowers that were not used to train it.



Accuracy = 85%: 17 of 20 flowers were correct. There are 9 correct class-0 calls and 8 correct class-1 calls.

Precision = 88.9%: 8 of 9 flowers called virginica really were virginica. **Recall = 80%:** 8 of 10 real virginica flowers were found.

F1 = 84.2%. One versicolor flower was wrongly called virginica, and two virginica flowers were missed. Class 1 is virginica for these metrics.

Only 20 flowers were tested. One changed prediction moves accuracy by 5 percentage points.

DAY 07 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie